

Binary Search + BinaryTree

Binary Search, Binary Tree,
Binary Search Tree,
are ALL different !

Binary Search

- Applied to sorted array, or any other random access media.
- Average case time Complexity = $\Omega(\log n)$
- Proven earlier.
- Worst case time Complexity = $O(\log n)$
- Why?

Recursive Algorithm

- `BinarySearch(i, j)`
- `if i > j`
- `return "unfound";`
- `else`
- `mid =  $\lfloor (i+j) / 2 \rfloor$ ;`
- `if A[mid] = target`
- `return mid;`
- `if A[mid] > target`
- `return BinarySearch(i, mid-1);`
- `else return BinarySearch(mid+1, j);`

Define $T(n)$

- $T(n)$ is the time complexity of $\text{BinarySearch}(i, j)$, where n is $j-i+1$.
- Then, both $\text{BinarySearch}(i, \text{mid}-1)$ and $\text{BinarySearch}(\text{mid}+1, j)$ take $T(\lfloor n/2 \rfloor)$ time.
- Hence, **by direct counting**, $T(n) = T(n/2) + 1$
- Case 2 applies.
- $T(n) = \Theta(\log n)$

Iterative algorithm

- `i=1; j= n;`
- `While i <= j do`
- `mid =  $\lfloor (i+j) / 2 \rfloor$ ;`
- `if A[mid] = target`
- `return mid;`
- `if A[mid] < target`
- `i = mid + 1;`
- `else j = mid - 1;`
- `Return "unfound";`

How to analyze the iterative algorithm?

- For each iteration, the time complexity is $O(1)$.
- Then, time complexity = #iteration.
- Count the number of iterations by **Indirect counting**.
- At the beginning of the
- 1st iteration, in $A[i \dots j]$, there are n elements.
- 2nd iteration,there are $\lfloor n/2 \rfloor \leq n/2$ elements.
- 3rd iteration,there are $\leq n/4$ elements.
- ...
- kth iteration,there are $\leq n/2^k$ elements.
- Since $i \leq j$, $n/2^k \geq 1$. What is the maximum value of k ?
- $\log n$.

Any implication of $O(\log n)$
time?

Binary Tree

Preorder Traversal - Revisit

- Preorder traversal
- $O(n)$ -time, $n = \text{\#node}$
- ```
void preorder(node* ptr) {
```
- ```
    if (ptr==NULL)
```
- ```
 return;
```
- ```
    cout << ptr->element << " ";
```
- ```
 preorder(ptr->left);
```
- ```
    preorder(ptr->right);
```
- ```
 return;
```
- ```
}
```
- Why is the time complexity $O(n)$?
- $T(n) = T(nL) + T(nR) + 1$
- Claim: $T(n) < cn$, $c > 1$.
- $T(n) = T(nL) + T(nR) + 1$
- $< c nL + c nR + 1$
- $< c nL + c nR + c$
- $= c(nL + nR + 1)$
- $= c n$

height

- If T is empty, $\text{height}(T)=0$;
 - If T is not empty, then
 - $\text{height}(T) = \max(\text{height}(TL), \text{height}(TR)) + 1$
-
- $\max(a,b) = a$ if $a \geq b$; b if $a < b$.

Algorithm height(T)

height(T)

 If T is empty

 return 0;

 hL=height(TL);

 hR=height(TR);

 return max(hL,hR)+1;

Algorithm size(T)

size(T)

 If T is empty

 return 0;

 nL=size(TL);

 nR=size(TR);

 return nL + nR + 1;

height(T) and size(T) are of the same structure.

Full binary tree

- There are many equivalent definitions for full binary tree.
 - The basic idea: Every level is fully filled up.
 - Level 1: one node
 - Level 2: 2 nodes
 - Level 3: 4 nodes
 - ...
 - Level h : 2^{h-1} nodes
-
- Summing up, we have
 - for an n -node full binary tree of height h , $n=2^h-1$.

Another definition

- For an n -node binary tree of height h , $n=2^h-1$ if and only if it is full. (can be proven, not in this course)
- Simple algorithm to test whether a binary tree is full or not.
- Input: a binary tree T
- $n = \text{size}(T)$;
- $h = \text{height}(T)$;
- If $n=2^h-1$, output “full”; else output “not full”;

Implication of this simple algorithm

- Mathematics is essential for designing simple and efficient algorithm.
- We need to know a recursive definition for height, a recursive definition for size, and the sufficient and necessary condition $n=2^h-1$ for fullness.

Some philosophical points for your interest only

- A myth: Complexity of ideas reflects our talent.
- Maybe true, but certainly not always true.
- Simplicity sometimes requires more on our talent.
- Simplicity does not mean trivial.
- Such a simple algorithm contains many theories behind.

Another property of binary tree

- Leaf node is a node without children.
- If every internal node has two children, then there is at least one internal node having two children as leaf nodes.
- If every internal node has two children,
 $\text{\#leaf-node} = \text{\#internal-node} + 1$.

- If every internal node has two children, then there is at least one internal node having two children as leaf nodes.
- Proof: Pick an internal node X at the lowest possible level.
- Then, none of the two children of X is an internal node; otherwise, X is not an internal node of the lowest possible level.
- Then, the two children of X must be leaf nodes.

- If every internal node has two children,
 $\#leaf-node = \#internal-node + 1$.
- Proof: Use induction on $\#internal-node$.
- Base case: $\#internal-node = 1$.
- The internal node has two children, which are leaf nodes. $\#leaf-node = 2$.
- The statement is true for the base case.
- Assume that the statement is true for $\#internal-node = k \geq 1$.

- Consider an arbitrary binary tree where $\# \text{internal-node} = k+1$, and any internal node has two children.
- Pick a internal node which has two leaf children nodes.
- Remove the leaf children nodes, and the internal node becomes a leaf node.
- Now, the $\# \text{internal-node} = k$.
- By inductive hypothesis, the $\# \text{leaf-node} = k+1$.
Now, put back the removed leaf nodes, the $\# \text{leaf-node} = k+1 - 1 + 2 = k + 2$.
- Hence, the original binary tree has $k+2$ leaf nodes.
- Done.
- Result from induction.

Application of Tree:

Lower bound for comparison-based sorting algorithms

How fast can we sort?

All the sorting algorithms we have seen so far are *comparison sorts*: only use comparisons to determine the relative order of elements.

- *E.g.*, insertion sort, merge sort, quicksort, heapsort.

The best worst-case running time that we've seen for comparison sorting is $O(n \lg n)$.

Is $O(n \lg n)$ the best we can do?

Decision trees can help us answer this question.